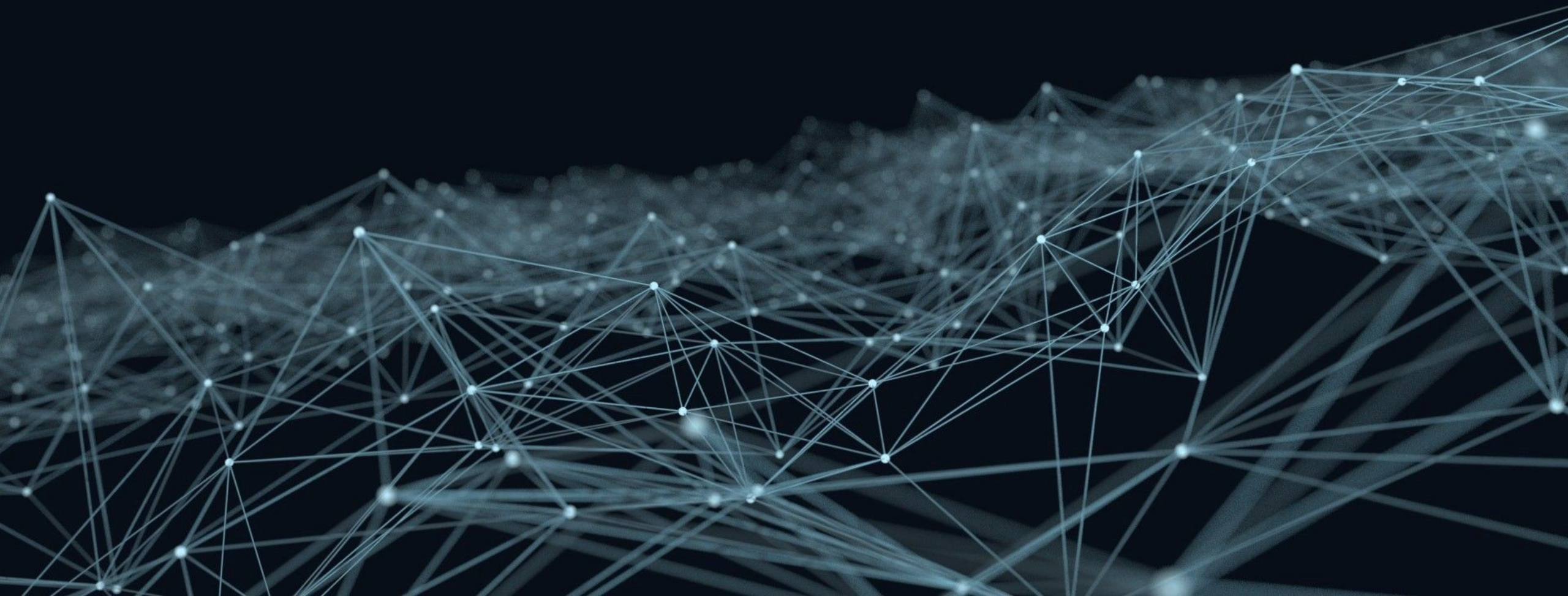
Java Full Stack Development

2. Spring Boot Architecture and Setup



Module Topics

- The Spring Framework
- The IoC container
- Beans and Configuration
- Package Structure
- Layered Design
- Spring Boot Configuration
- Spring Profiles
- Spring Boot Lifecycle

The Java Enterprise Problem (Pre-2003)

- Enterprise Java ran on J2EE
  - Heavyweight, server-managed component model
  - Core building block was the EJB (Enterprise JavaBean)
    - EJBs are owned and managed by the application server
  - Consequences
    - Massive XML configuration files
    - Mandatory boilerplate interface hierarchies
    - Unit testing was effectively impossible because every component needed a running server
    - Business logic buried under infrastructure ceremony

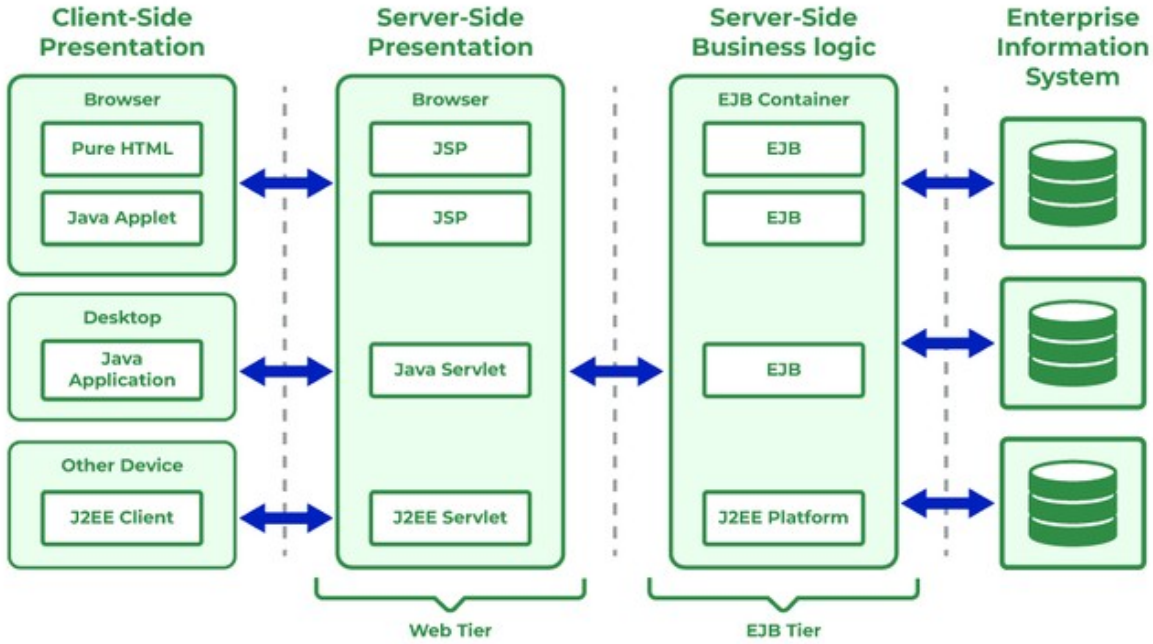


Image Credit: https://www.geeksforgeeks.org/java/j2ee-multitier-architecture/

Spring Framework (2003)

- The IoC Revolution
  - Based on the idea that the application server should not own the developer's objects
  - Introduced the IoC (Inversion of Control) container
    - A lightweight runtime inside the application
  - The container's responsibility
    - To create objects, resolve dependencies, manage lifecycles of Java components
  - Developer writes plain Java classes (POJOs)
    - The container wires them together
  - Results in testable, loosely coupled code with no server dependency

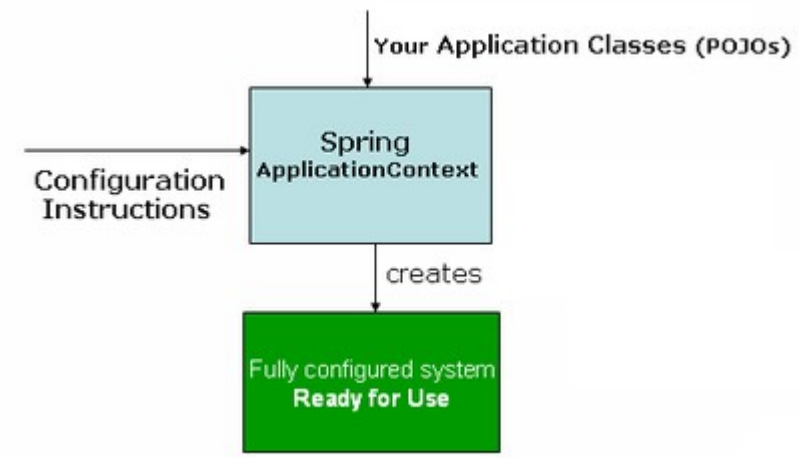


Image Credit: https://www.dariawan.com/tutorials/spring/core-concepts-of-the-spring-framework/

Spring Configuration Issues

- Spring solved EJB complexity
- But introduced the problem of XML configuration bloat
  - A production Spring application required hundreds of lines of XML
  - Data sources, transaction managers, security filters, MVC dispatchers all had to be declared in XML
  - XML had to be kept in sync with Java code manually
  - This is the configuration complexity problem Spring Boot was built to solve

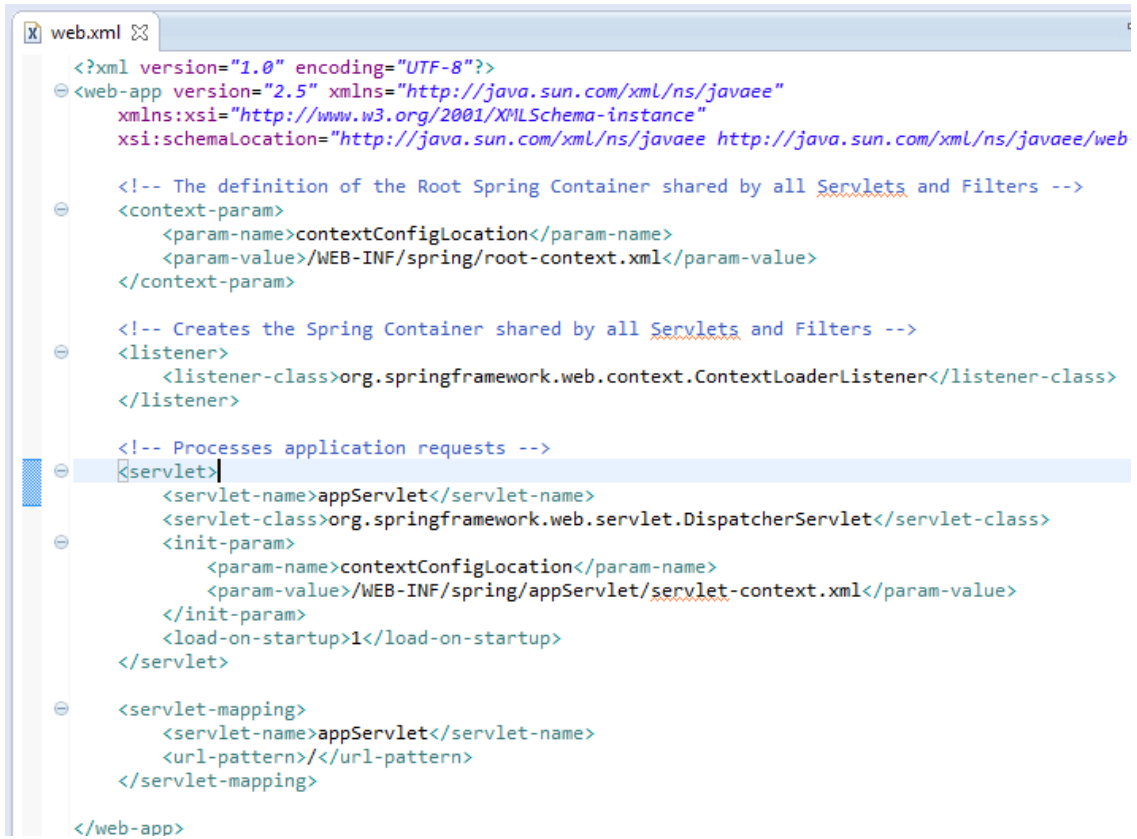


Image Credit: https://www.dariawan.com/tutorials/spring/core-concepts-of-the-spring-framework/

Spring Boot (2014)

- Convention over configuration
  - Introduced opinionated defaults
  - The framework makes sensible decisions so you developers don't have to
    - Adding a JDBC driver results in Spring Boot creating a DataSource automatically
    - Adding spring-boot-starter-web automatically adds a ready to go embedded Tomcat server and DispatcherServlet
  - Every default is can be overridden
    - Developers only configure what differs from the default convention
  - Developer express decisions instead of writing a lot of boilerplate configuration

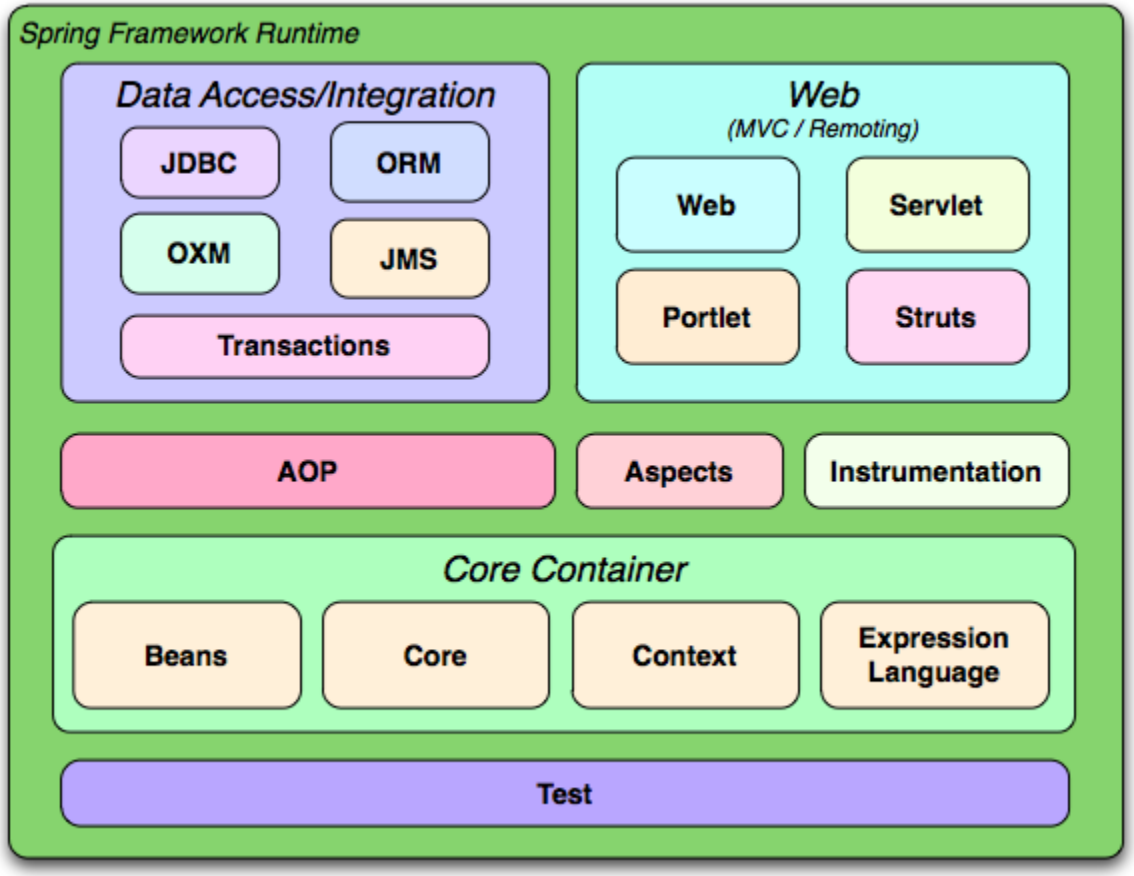


Image Credit: https://docs.spring.io/spring-framework/docs/3.0.x/spring-framework-reference/html/overview.html

Spring Boot

- Spring Boot is a layer built on top of Spring components
  - Spring Boot is an opinionated launcher for Spring
  - Every Spring Boot application is a Spring application
- Spring Boot adds three things
  - Auto-configuration conditional bean wiring based on classpath contents
  - Starter dependencies: curated, tested dependency bundles organized by functionality
  - Embedded server: Tomcat/Jetty/Undertow packaged inside the JAR
- Spring Framework docs and Spring Boot docs are both relevant because they address different layers in the same ecosystem

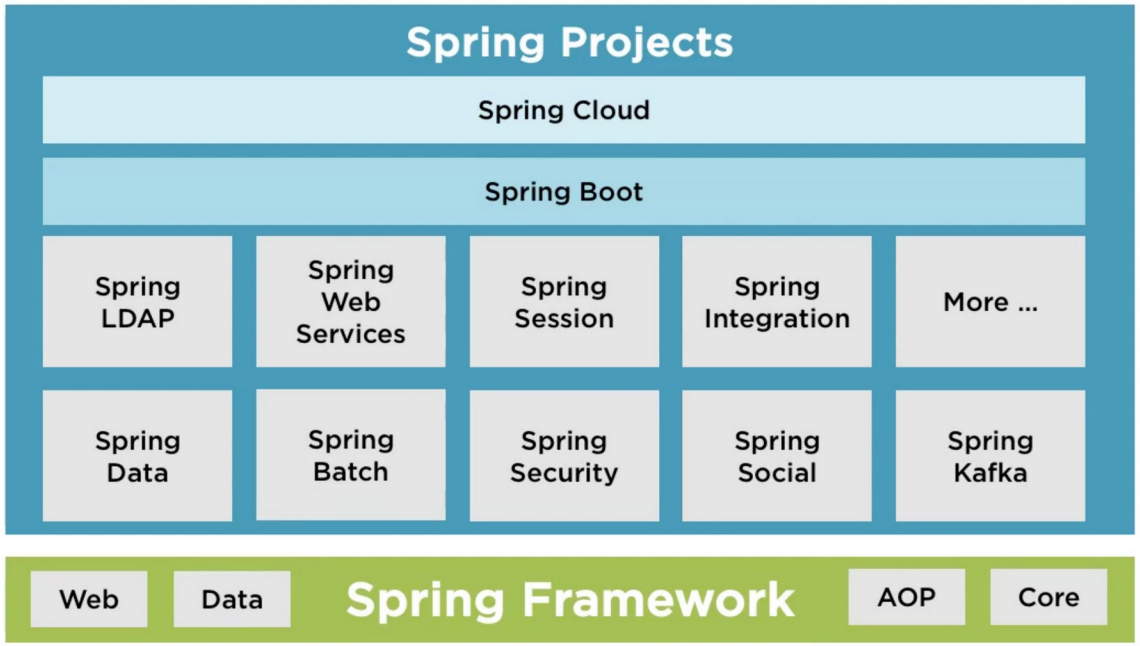


Image Credit: https://peerdh.com/blogs/programming-insights/understanding-the-spring-ecosystem-a-comprehensive-guide

The IoC Container

- IoC is a runtime that manages the complete lifecycle of a Spring application's objects
  - These managed objects are called beans
- The container is responsible for
  - Instantiating objects in the correct dependency order
  - Injecting dependencies at construction time
  - Managing initialization and destruction callbacks
  - Providing beans to anything that needs them, anywhere in the application
- Annotations express developer intentions
  - The container executes them

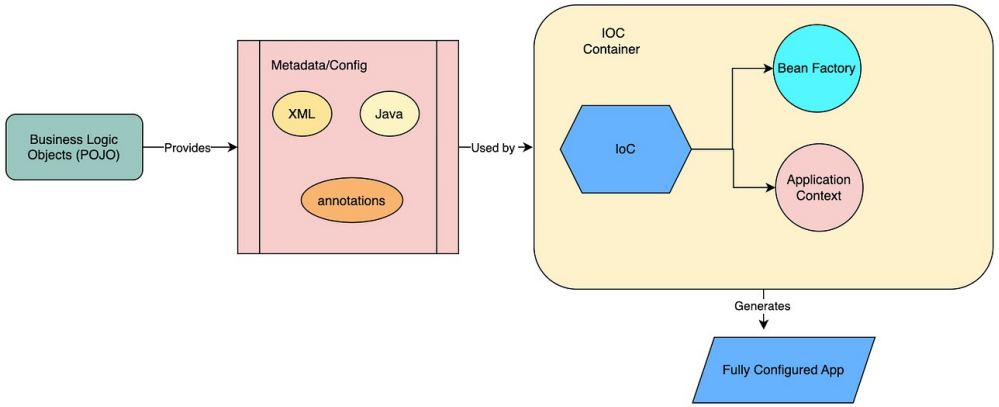


Image Credit: https://medium.com/@ramjasprankur/mastering-dependency-injection-and-ioc-container-in-spring-boot-a-comprehensive-guide-bc6d89c39c8c

Inversion of Control

- In traditional code
  - A class creates its own dependencies
  - Code is built from the top down
- IoC reverses the order
  - The container creates objects and pushes dependencies into objects that need them
  - Developer written classes become plain Java objects with no knowledge of how they are assembled
- This produces code that is testable without a server, loosely coupled, and replaceable

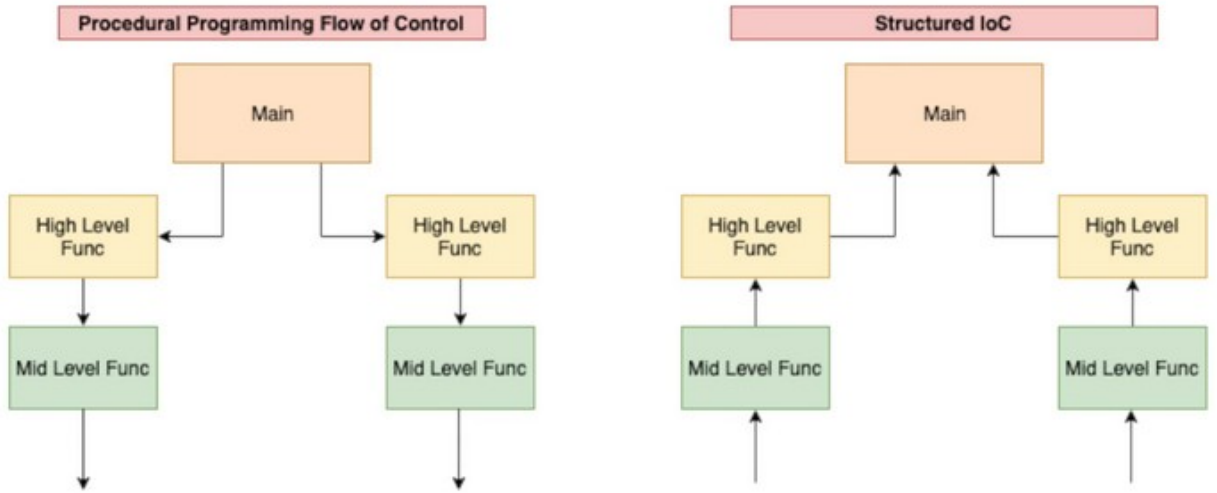


Image Credit: https://ead.adhikariarjun.com.np/design-patterns/di-and-ioc/

ApplicationContext

- BeanFactory
  - Basic IoC container with lazy bean creation and minimal overhead
- ApplicationContext
  - Extends BeanFactory and adds the enterprise features always needed like
    - Event Publication: beans can publish and listen to application events
    - AOP Integration: declarative security interceptors, and logging aspects are woven in transparently
    - Environment Abstraction: unified access to profiles, property sources, and the configuration precedence stack
  - In Spring Boot, code always works with an ApplicationContext
    - It is created automatically at startup

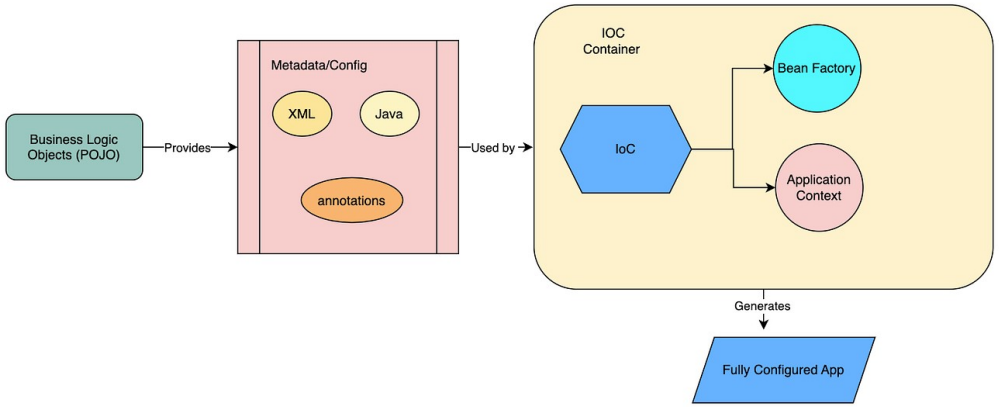
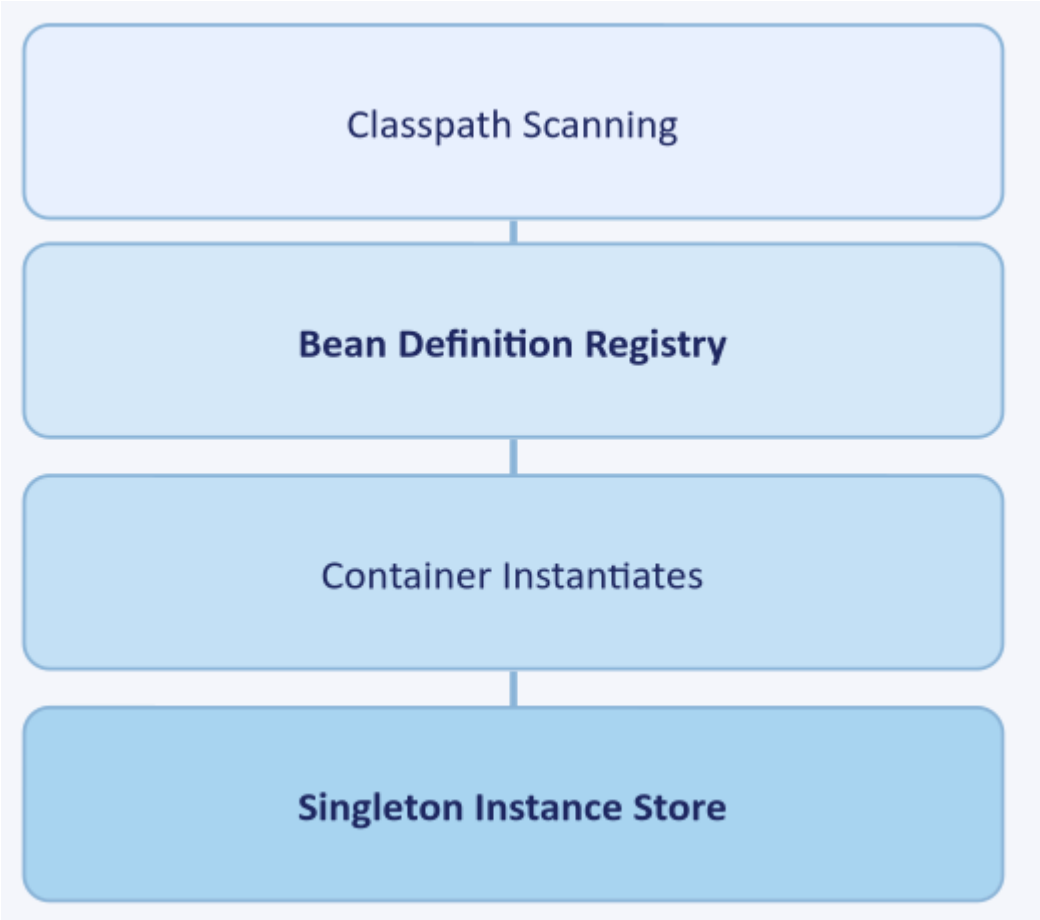


Image Credit: https://medium.com/@ramjasprankur/mastering-dependency-injection-and-ioc-container-in-spring-boot-a-comprehensive-guide-bc6d89c39c8c

Beans

- Bean definition
  - Describes the class, scope, dependencies, and lifecycle callbacks for an object
  - Registered during classpath scanning at startup, no instances are created
- Bean instances
  - Are then created from definitions
  - Either eagerly, as singletons at startup
  - Or lazily, where they are created on first request
- The container maintains an instance registry
  - This is a map of name/type and live objects
  - When a dependency is needed
    - The container checks the registry first
    - Only on not finding an entry does it create a new instance



Stereotype Annotations

- Spring discovers beans by scanning for stereotype annotations on the classpath
  - All stereotypes are specializations of @Component
  - Annotations differ in semantic meaning, not container behaviour, with the exception of @Repository (more on this later)

Annotation	Layer	What It Signals
`@Component`	Any	Generic Spring-managed component. Base stereotype — use when no more specific annotation fits
`@Service`	Business	Business logic class. No additional container behaviour — the distinction is semantic and enforced by convention
`@Repository`	Data access	Activates **exception translation** — raw JDBC/JPA exceptions are automatically converted to Spring's `DataAccessException` hierarchy
`@RestController`	Web	Marks an MVC controller; adds `@ResponseBody` to every method implicitly
`@Configuration`	Config	Declares `@Bean` factory methods. Its methods are intercepted by a CGLIB proxy to enforce singleton semantics

Explicit Wiring

- @Configuration
  - Used when a class cannot be annotated directly
    - For example, third-party library classes
- Each @Bean method is a factory
  - When the application starts, the container calls each @Bean method exactly once
  - The object returned is registered in the bean registry
  - From that point forward, any other bean that declares a dependency on that type receives this registered instance
    - The container never calls the @Bean method again.
  - The method name becomes the default bean name
    - Use @Bean("name") to override

```
// WebClient is a third-party class — we cannot add @Service to it
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient hrApiClient(EmployeeServiceConfig cfg) {
        return WebClient.builder()
                .baseUrl(cfg.baseUrl())
                .defaultHeader(HttpHeaders.ACCEPT, MediaType.APPLICATION_JSON_VALUE)
                .build();
    }
}
```

Bean Scopes

- Scope determines how many instances exist and for how long
  - singleton is correct for all stateless objects: services, repositories, configuration
  - prototype is correct for objects that hold per-call mutable state
  - request and session are rarely needed in REST APIs, which prefer stateless designs

Scope	Instances	Created	Container manages destruction?
`singleton` *(default)*	One per ApplicationContext	At startup, shared across all threads	Yes
`prototype`	New instance per request to container	On demand	**No** — caller is responsible
`request`	One per HTTP request	Per request (web context required)	Yes
`session`	One per HTTP session	Per session (web context required)	Yes

The Stateful Singleton Antipattern

- The singleton bean is shared across all threads simultaneously
  - Any mutable instance field is therefore shared state
    - Represents a potential race condition
  - Single most common Spring bug for developers new to the framework

```java
@Service
public class GreetingService {

    // ❌ Mutable instance field on a singleton
    private String lastGreeting;

    public String greet(String name) {
        lastGreeting = "Hello, " + name;
        return lastGreeting;
    }

    public String getLastGreeting() {
        return lastGreeting;
    }
}
```

java

```java
@Service
public class GreetingService {

    public String greet(String name) {
        // ✅ Local variable — lives on the call stack, invisible to other threads
        String greeting = "Hello, " + name;
        return greeting;
    }
}
```

The Prototype Pattern

- A bean may need a per-call mutable state
  - For example; a builder, a batch processor, a CSV exporter
- Declare it to have prototype scope
  - Inject from an ObjectProvider<T>
  - A lazy factory that creates a fresh instance on each getObject() call

```java
// ① Declare the stateful bean as prototype
@Component
@Scope("prototype")
public class CsvExportJob {
    private final List<Row> buffer = new ArrayList<>();

    public void addRow(Row row) { buffer.add(row); }
    public byte[] build() { /* serialise buffer to CSV bytes */ }
}

// ② Inject ObjectProvider into the singleton
@Service
public class ExportService {
    private final ObjectProvider<CsvExportJob> jobs;

    public ExportService(ObjectProvider<CsvExportJob> jobs) {
        this.jobs = jobs;
    }

    public byte[] export(List<Employee> employees) {
        CsvExportJob job = jobs.getObject(); // fresh prototype instance each call
        employees.forEach(e -> job.addRow(toRow(e)));
        return job.build();
    }
}
```

The Prototype Pattern

- ObjectProvider<T>
  - Spring interface that represents a lazy, optional handle to a bean in the container
  - A factory-like object that is used to retrieve or create the bean on demand, at the moment it is needed
- The simplest way to think about it is this
  - Normal constructor injection says "give me the bean now, at startup"
  - ObjectProvider<T> says "give me the ability to get the bean later, when I ask for it"

```java
// ① Declare the stateful bean as prototype
@Component
@Scope("prototype")
public class CsvExportJob {
    private final List<Row> buffer = new ArrayList<>();

    public void addRow(Row row) { buffer.add(row); }
    public byte[] build() { /* serialise buffer to CSV bytes */ }
}

// ② Inject ObjectProvider into the singleton
@Service
public class ExportService {
    private final ObjectProvider<CsvExportJob> jobs;

    public ExportService(ObjectProvider<CsvExportJob> jobs) {
        this.jobs = jobs;
    }

    public byte[] export(List<Employee> employees) {
        CsvExportJob job = jobs.getObject(); // fresh prototype instance each call
        employees.forEach(e -> job.addRow(toRow(e)));
        return job.build();
    }
}
```

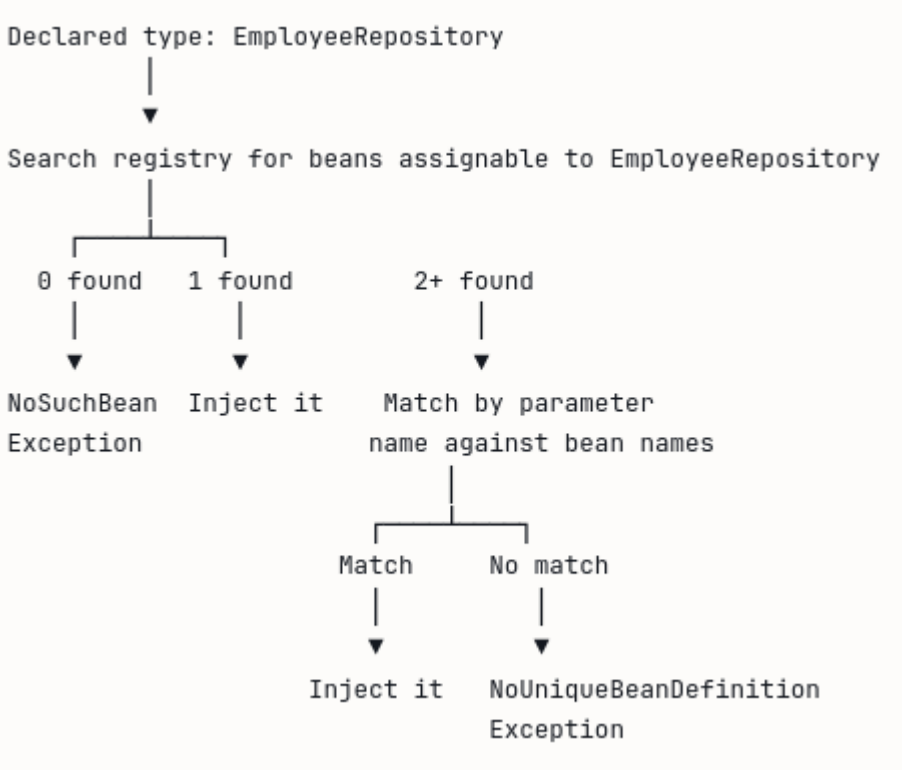
The Bean Lifecycle

Phase	What Happens	Your Hook
1. Instantiate	Constructor called. Constructor-injected dependencies resolved recursively	Constructor
2. Populate	`@Autowired` fields and setter injection applied (if used)	—
3. Aware interfaces	Container injects itself if the bean implements `BeanNameAware`, `ApplicationContextAware`, etc.	Implement interface
4. `@PostConstruct`	Runs after all injection is complete — the bean is fully assembled	`@PostConstruct` method
5. In Use	Bean serves requests and handles calls	—
6. `@PreDestroy`	Runs before the context closes — release resources	`@PreDestroy` method

```java
@Service
public class CacheWarmingService {

    private final EmployeeRepository repo;
    private final Map<Long, Employee> cache;

    public CacheWarmingService(EmployeeRepository repo, Map<Long, Employee> c
        this.repo = repo;
        this.cache = cache;
    }

    @PostConstruct
    void warmCache() {
        // Runs after injection — repo is guaranteed non-null here
        repo.findAll().forEach(e -> cache.put(e.id(), e));
    }

    @PreDestroy
    void evictCache() {
        // Runs before the context closes — clean up resources
        cache.clear();
    }
}
```

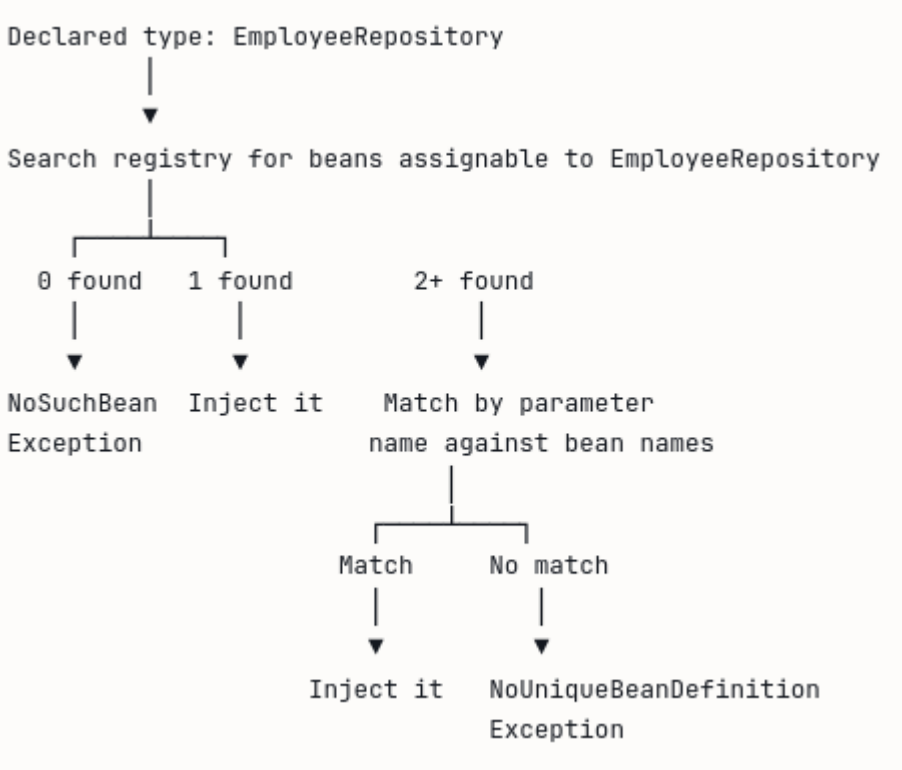
Autowiring

- The process used by the Spring container
  - Automatically resolves and injects a bean's dependencies without having to write code
    - Programmer declare what a class needs
    - The container figures out which bean satisfies that need and provides it
  - Autowiring happens at application startup
    - If the container cannot satisfy a dependency, the application fails immediately with a clear error
    - The application does not start with a broken object graph to avoid failing later at runtime



Autowiring

- Process when injecting a dependency.
  - Step 1: Type matching
    - Container looks in its registry for all beans that are assignable to the declared type.
    - For EmployeeRepository repo, it looks for any bean that implements or extends EmployeeRepository
  - Step 2: Name matching
    - If exactly one match is found, it is injected.
    - If multiple matches are found, the container looks at the parameter name and tries to match it against a bean name to break the tie
  - If the container finds zero matches
    - it throws NoSuchBeanDefinitionException at startup
  - If it finds multiple matches and cannot break the tie
    - It throws NoUniqueBeanDefinitionException at startup



Injection Styles

- Spring supports three styles of injection
- Only one is recommended for production code

Style	How it looks	Verdict
Constructor injection	Dependency declared as constructor parameter	✅ Always use this
Setter injection	Dependency set via a `@Autowired` setter method	⚠️ Only for optional dependencies
Field injection	`@Autowired` directly on the field	❌ Never use in production code

Injection Styles

- Constructor injection
  - The dependency is declared as a parameter on the constructor
    - if there is exactly one constructor, @Autowired is implicit and no annotation is required
  - The object is fully initialized the moment the constructor completes
    - There is no window where a dependency is null
  - Fields can be declared final, enforcing immutability after construction
  - With implicit autowiring
    - The class can be instantiated and tested with plain Java, no Spring context required

```java
@Service
public class EmployeeService {

    private final EmployeeRepository repo;
    private final AuditService auditService;

    // @Autowired not required — single constructor is implicit injection point
    public EmployeeService(EmployeeRepository repo, AuditService auditService) {
        this.repo = repo;
        this.auditService = auditService;
    }
}
```

```java
// In a unit test — no Spring context needed at all
EmployeeService service = new EmployeeService(mockRepo, mockAudit);
```

Injection Styles

- Field injection
  - Field injection places @Autowired directly on a private field
- Creates four concrete problems
  - Hidden dependencies
    - No way to know what this class needs without reading its internals
    - A constructor makes dependencies part of the public contract of the class
  - Untestable without a container

```
// ❌ Field injection — never use this
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repo;

    @Autowired
    private AuditService auditService;
}
```

Injection Styles

- Creates four concrete problems
  - Fields cannot be final
    - Injection happens after construction, so final is not possible
    - The fields can be reassigned at any point, and the compiler cannot catch it
  - Broken object construction guarantee
    - Between the moment the JVM constructs the object and the moment Spring injects the fields, the object exists in an invalid state
    - Any code that runs in that window, such as a superclass constructor, will encounter null fields

```
// ❌ Field injection — never use this
@Service
public class EmployeeService {

    @Autowired
    private EmployeeRepository repo;

    @Autowired
    private AuditService auditService;
}
```

Injection Styles

- Setter injection
  - Places @Autowired on a setter method
  - Appropriate in exactly one situation
  - Dependency is genuinely optional
  - The class can operate correctly when the dependency is absent

```java
@Service
public class ReportService {

    private final MandatoryReportRepository repo;      // required
    private NotificationClient notificationClient;     // optional

    public ReportService(MandatoryReportRepository repo) {
        this.repo = repo;
    }

    @Autowired(required = false)
    public void setNotificationClient(NotificationClient client) {
        this.notificationClient = client;
    }

    public void generateReport(ReportRequest request) {
        Report report = repo.build(request);
        if (notificationClient != null) {
            notificationClient.send(report.summary());
        }
    }
}
```

Autowiring Summary

- Autowiring is type-driven
  - The container matches by type first, then by name to break ties
- Use constructor injection for all required dependencies
  - It is the only style that produces a fully initialized, testable, immutable object
- Use setter injection only for genuinely optional dependencies
  - Prefer ObjectProvider<T> as a better choice
- Never use field injection
  - It hides dependencies, prevents final, and makes the class untestable without a container
  - Inject interfaces, not concrete classes
    - This is what decouples the layers and enables substitution
- Autowiring failures are startup failures

Auto-Configuration

- The problem auto-configuration solves
  - Every application that uses a database needs the same connection pool setup
  - Every application that uses Kafka needs the same producer and consumer configuration
  - Developers were writing identical boilerplate across thousands of projects
    - Adding a library was never enough
    - Developers also had to write configuration code to make it usable
- Spring Boot's answer
  - If you have added a library, you almost certainly want to use it in the standard way
  - Then the framework configures it for you automatically

Auto-Configuration

- What auto-configuration does
  - The moment Spring Boot detects a library on the classpath, it configures it with sensible defaults
    - For example, if the Oracle JDBC driver is added and a URL is provided
    - Spring Boot creates a fully configured connection pool automatically
  - If the web starter is added
    - An embedded Tomcat server starts automatically
  - If the security starter is added
    - All endpoints are protected by default
  - If the Kafka starter is added
    - A KafkaTemplate is available for injection immediately
  - In all cases, no configuration code is needed

Auto-Configuration

- Gating mechanisms
  - Conditions are annotations
    - They query the state of the application before applying any configuration
  - If any condition fails, the entire configuration class is skipped as if it did not exist
- What Spring Boot actually does when it detects a JDBC driver
  - The two conditions read
    - Create a DataSource
    - But only if a JDBC driver is present
    - And only if the application has not already created its own DataSource

Condition Annotation	The Question It Asks
`@ConditionalOnClass`	Is this class present on the classpath?
`@ConditionalOnMissingBean`	Has the application already defined a bean of this type?
`@ConditionalOnProperty`	Is this property set in `application.yml`?
`@ConditionalOnWebApplication`	Is this a web application context?

```
@AutoConfiguration
@ConditionalOnClass(DataSource.class)       // only if a JDBC driver is on the class
@ConditionalOnMissingBean(DataSource.class) // only if the app hasn't defined its ow
public class DataSourceAutoConfiguration {

    @Bean
    public DataSource dataSource(DataSourceProperties properties) {
        // builds a connection pool from the url, username, password in application.
    }
}
```

Auto-Configuration

- @ConditionalOnMissingBean
  - This single condition is what makes auto-configuration safe and non-intrusive
  - The moment the written code defines a @Bean of the same type
    - The auto-configuration backs off entirely
    - The coded bean definition is used
  - Simply provide a definition for a bean and the default is never created

```
// This is all it takes to replace the auto-configured DataSource entirely
@Bean
public DataSource dataSource() {
    HikariDataSource ds = new HikariDataSource();
    ds.setJdbcUrl("jdbc:oracle:thin:@host:1521/PROD");
    ds.setMaximumPoolSize(20);
    ds.setConnectionTimeout(3000);
    return ds;
}
```

Auto-Configuration

- Overriding auto-configuration
  - Defining a replacement bean
    - Covered in the previous slide
  - Set properties file
    - Either a properties file or yaml file in the resources directory
  - Exclude explicitly
    - Remove an entire auto-configuration class
    - Use when you want none of its beans, not just overrides.

properties

```properties
# application.properties style
server.port=8080
spring.application.name=employee-service
```

yaml

```yaml
# application.yml style — same settings
server:
  port: 8080
spring:
  application:
    name: employee-service
```

```
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
public class App { ... }
```

Auto-Configuration

- For most of a typical application you never think about auto-configuration at all
  - Add a dependency, set a few properties, and everything works
  - That is the intended experience
- You only engage with it directly when the default is not what you need
  - Examples
    - Custom connection pool sizing
    - Non-standard security configuration
    - Specialized Kafka producer setup
  - Three override options
    - Provide your own bean
    - Set a property
    - Exclude the class

Diagnosing Auto-Configuration

- The actuator conditions report is used to diagnose broken auto-configurations
- Shows every auto-configuration class Spring Boot evaluated
  - Whether it was applied
  - And if not, which condition caused it to be skipped
- The actuator has to be included in the pom file as a dependency

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Diagnosing Auto-Configuration

- Access from the running application URL
  - By default Spring Boot only exposes /actuator/health over HTTP.
  - This is a deliberate security decision
  - You explicitly opt in to exposing others
    - Set in the configurations file
    - Either yaml or properties format

Endpoint	What It Shows
`/actuator/health`	Whether the application and its dependencies are up
`/actuator/conditions`	Which auto-configurations fired and which were skipped, and why
`/actuator/beans`	Every bean in the application context and its dependencies
`/actuator/env`	All configuration properties and where each value came from
`/actuator/metrics`	Runtime metrics — memory, threads, HTTP request counts
`/actuator/mappings`	Every HTTP endpoint the application exposes

```yaml
management:
  endpoints:
    web:
      exposure:
        include: health, conditions, beans, env, metrics, mappings
```

```
management.endpoints.web.exposure.include=health,conditions,beans,env,metrics,mappings
```

Package Structure

- In Spring Boot
  - The package layout is where architectural decisions are made explicit and enforced
  - Every package boundary in a well-structured Spring Boot application answers specific questions
    - Who is responsible for this behaviour?
    - What is this component allowed to know about?
    - In which direction do dependencies flow?
    - Can this component be tested in isolation?
  - Dependencies flow inward
    - Outer layers know about inner layers
    - Inner layers know nothing about outer layers
    - Software engineering principle: the dependency rule from layered architecture
    - Often referred to as the Dependency Inversion Principle

Standard Layered Package Layout

- A Spring Boot application is organized into a set of well-defined packages, each with a single, bounded responsibility
  - The controller knows about the service
  - The service knows about the repository and the domain
  - The repository knows about the domain
  - Nothing flows backwards
    - The controller has no direct knowledge of the repository.
    - The repository has no knowledge of HTTP

```
com.example.app
├── controller      ← HTTP boundary: handles requests and responses
├── service         ← Business logic: the core of what the application does
├── repository      ← Data access: reads and writes to the database
├── domain          ← Data model: the persistent entities (database-mapped
objects)
├── dto             ← API shapes: what clients send and receive
├── config          ← Configuration: wires infrastructure beans
└── exception       ← Error model: custom exceptions and global error handling
```

```
controller  →  service  →  repository  →  domain
                 ↓
               domain (also used by service for entity operations)
                 ↓
               dto (used by controller and service for API translation)
```

Standard Layered Package Layout

- The controller layer
  - HTTP boundary of the application.
  - Spring MVC routes incoming HTTP requests to controller methods
    - Via annotations like @GetMapping and @PostMapping
  - Receives HTTP requests and extracts input
    - Path variables, request bodies, query parameters
  - Validates that input is structurally correct using @Valid
  - Calls the service layer with that input
  - Translates the service result into an HTTP response

```java
@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDto> getById(@PathVariable Long id) {
        // Delegate immediately — no logic here
        return ResponseEntity.ok(employeeService.findById(id));
    }
}
```

Standard Layered Package Layout

- The controller layer should not
  - Contain any business logic
    - Business logic belongs in the service
  - Call repositories directly
    - Repository access must go through the service
  - Hold any mutable state
    - Controllers are singletons shared across all requests

```java
@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDto> getById(@PathVariable Long id) {
        // Delegate immediately — no logic here
        return ResponseEntity.ok(employeeService.findById(id));
    }
}
```

Standard Layered Package Layout

- The service layer
  - Business logic core of the application
  - Where decisions are made, rules are enforced, and workflows are orchestrated
  - Implements all business rules
  - Owns transaction boundaries
    - @Transactional lives here, not in the controller or repository
  - Coordinates calls across multiple repositories or external services
  - Translates between domain entities and DTOs

```java
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    @Transactional(readOnly = true)
    public EmployeeDto findById(Long id) {
        Employee employee = employeeRepository.findById(id)
            .orElseThrow(() -> new EmployeeNotFoundException(id));
        return toDto(employee);   // translation happens here
    }
}
```

Standard Layered Package Layout

- The service layer should not
  - Know anything about HTTP
    - HttpServletRequest,and ResponseEntity should never be referred to
  - Handle JSON serialisation
    - Done by Jackson and the controller's

```java
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    @Transactional(readOnly = true)
    public EmployeeDto findById(Long id) {
        Employee employee = employeeRepository.findById(id)
            .orElseThrow(() -> new EmployeeNotFoundException(id));
        return toDto(employee);   // translation happens here
    }
}
```

Standard Layered Package Layout

- The repository layer
  - The data access layer
  - In Spring Data JPA, this is typically an interface that extends JpaRepository
  - Spring generates the implementation at startup.
  - Executes queries against the database and nothing else.
- Must not
  - Contain business logic
  - Throw service-level exceptions
  - Reference HTTP concepts

```java
@Service
public class EmployeeService {

    private final EmployeeRepository employeeRepository;

    public EmployeeService(EmployeeRepository employeeRepository) {
        this.employeeRepository = employeeRepository;
    }

    @Transactional(readOnly = true)
    public EmployeeDto findById(Long id) {
        Employee employee = employeeRepository.findById(id)
            .orElseThrow(() -> new EmployeeNotFoundException(id));
        return toDto(employee);   // translation happens here
    }
}
```

```java
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByDepartmentId(Long departmentId);
}
```

Domain and DTO

- The domain layer
  - JPA entity classes
  - Java objects that are directly mapped to database tables
  - The @Entity annotation tells Hibernate to manage these objects and synchronise them with the database
- Represents the persistent data model
  - It is the database's view of your data

```java
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String firstName;
    private String lastName;
    private String passwordHash;      // sensitive — must never reach the API
    private String internalStatusCode; // internal — must never reach the API

    @ManyToOne(fetch = FetchType.LAZY)
    private Department department;
}
```

Domain and DTO

- The DTO layer
  - Plain data carrier objects
  - Java records (preferred) or classes
  - Define exactly what the API receives from clients and returns to clients
- Represents the API's view of the data
  - Only the fields that clients are permitted to see or send

```
public record EmployeeDto(
    Long id,
    String firstName,
    String lastName,
    String departmentName
    // no passwordHash – never exposed
    // no internalStatusCode – never exposed
) {}
```

Domain and DTO

- Returning a JPA entity directly from a REST endpoint instead of a DTO causes four problems
- Uncontrolled serialisation and N+1 queries
  - Jackson (the JSON serialiser) will traverse and serialize the entire object graph of an entity
  - If the Employee has a lazy-loaded Department, and Department has a lazy-loaded list of Employees, Jackson will trigger Hibernate to load everything
  - This happens outside any transaction boundary
  - Problematic if the transaction has already closed
  - The result is either a LazyInitializationException crash or a cascade of unintended database queries

Domain and DTO

- Database schema changes break your API
  - If your entity field is called empSurname in the database and is exposed the entity directly
    - Then the API field is empSurname
  - If the column name changes, then the public API contract also changes
    - Every client breaks
  - With a DTO, the API field name is defined in the DTO
    - Allows the database to change without affecting clients

Domain and DTO

- Sensitive fields are inadvertently exposed
  - The entity exists to represent a database row
  - Database rows contain things clients should never see
    - Password hashes, internal status codes, audit timestamps, soft-delete flags
  - If the entity is returned, Jackson serializes all of unless you remember to annotate every sensitive field with @JsonIgnore
  - A common error is to forget the annotation as the entity evolves

Domain and DTO

- Unexpected database writes
  - Hibernate tracks changes to managed entity instances, called dirty checking
  - If a client sends a JSON request body and it is deserialized directly into a managed @Entity object
  - Then Hibernate may detect the "changes" and write them back to the database at the end of the transaction
  - Even if save() is never called
  - Creates a subtle, dangerous bug that is difficult to reproduce

Service Layer and Translation

- The service layer is where entities and DTOs are translated into each other
- This translation is a deliberate, explicit decision
  - This is not boilerplate to be removed or automated
- Inbound
  - Client sends data → service persists it

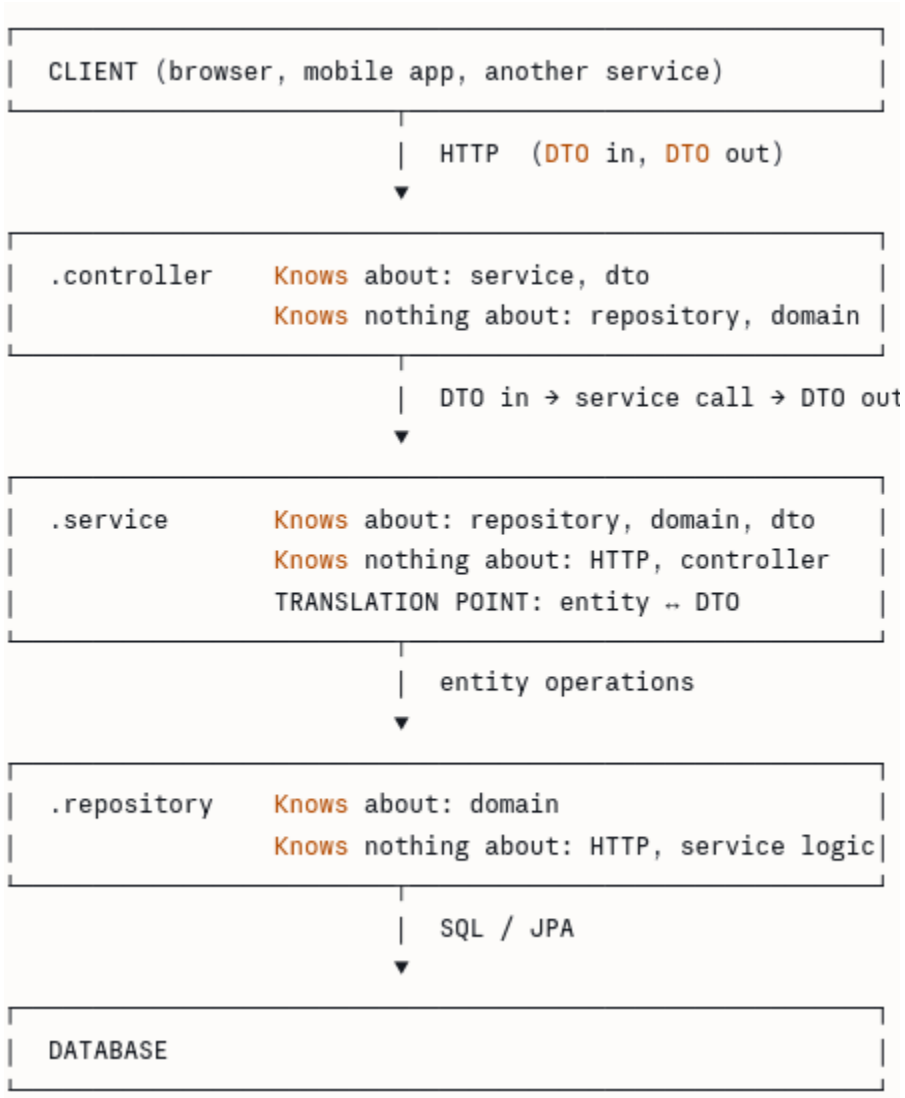
```java
// Client sends a CreateEmployeeRequest DTO
// Service maps it to an Employee entity before persisting

@Transactional
public EmployeeDto createEmployee(CreateEmployeeRequest request) {

    // Explicit field-by-field mapping — deliberate decision on every field
    Employee employee = new Employee();
    employee.setFirstName(request.firstName());
    employee.setLastName(request.lastName());
    employee.setDepartmentId(request.departmentId());
    // passwordHash is set here from a separate hashing step — not from the request
    // internalStatusCode is set by business logic — not from the request

    Employee saved = employeeRepository.save(employee);
    return toDto(saved);
}
```

Service Layer and Translation

- Every field that crosses the API boundary is an explicit decision visible in this code
  - If a developer adds a sensitive field to the entity, it does not automatically appear in the API
  - It only appears when a developer makes the conscious choice to add it to the DTO and to the mapping method
- Outbound
  - service reads entity → client receives DTO

```
// Private mapping method — owned by the service
private EmployeeDto toDto(Employee employee) {
    return new EmployeeDto(
        employee.getId(),
        employee.getFirstName(),
        employee.getLastName(),
        employee.getDepartment().getName()
        // passwordHash deliberately omitted
        // internalStatusCode deliberately omitted
    );
}
```

The Dependency Rule



Common Violations

Violation	Immediate Consequence	Long-term Consequence
Controller calls repository directly, bypassing service	`@Transactional` boundaries are not enforced	Business logic spreads across layers; impossible to test without a web context
Entity returned directly from controller	Sensitive fields exposed; N+1 queries at serialisation time	Schema changes break the API; clients depend on internal structure
`@Transactional` placed on a controller method	Transaction scope includes HTTP deserialisation overhead; errors in response marshalling trigger rollback	Unpredictable transaction behaviour under load
Service imports `HttpServletRequest` or `ResponseEntity`	Service cannot be tested without a mock HTTP context	Service logic becomes permanently coupled to the web layer
Repository throws business exceptions (`InvalidEmployeeStateException`)	Repository has knowledge of business rules it should not own	Two places now define the same business rule; they will diverge

Configuration Management

- Configuration is a security and compliance boundary
  - Consider what lives in configuration
    - Database credentials
    - OAuth2 client secrets
    - API keys for external services
    - Feature flags that control which business rules are active
    - Service endpoint URLs that determine where data is sent
  - Every one of those values differs between environment
  - Every one of them must not be hardcoded in source
  - Several of them must not appear in source control at all
    - Even in encrypted form in many regulated environments

Configuration Management

- Spring Boot provides a sophisticated configuration model with
  - Multiple sources
  - A defined precedence order
  - Strong typing.
- That model provides the mechanism
  - Requires a well designed configuration strategy to use the model effectively
  - The three requirements a configuration strategy must satisfy
    - Environment parity: the same application artifact (JAR file) runs in every environment; only the configuration changes
    - Secret hygiene: secrets are never in source control, never in logs, never in build artifacts
    - Fast-fail: a misconfigured application crashes at startup, not partway through a production transaction

Configuration Precedence Stack

- Spring Boot resolves configuration values from multiple sources simultaneously
  - When the same property is defined in more than one place
  - A strict precedence order determines which value wins
  - Secrets belong at priority 3
    - Injected at runtime by a secrets manager such as HashiCorp Vault or AWS Secrets Manager
    - They never touch the filesystem
    - They never appear in a YAML file
    - They never enter source control

Priority	Source	Typical Use
1 (Highest)	Command-line arguments (`--key=value`)	Ad-hoc overrides during development and debugging
2	`SPRING_APPLICATION_JSON` environment variable	Container-level JSON config injection
3	OS environment variables (e.g. `SPRING_DATASOURCE_URL`)	Secret and credential injection from a secrets manager
4	`application-{profile}.yml`	Environment-specific defaults committed to source control
5 (Lowest)	`application.yml`	Universal defaults shared across all environments

Configuration Mechanisms

- Spring Boot provides two mechanisms for reading configuration into application code
  - They are not equivalent and are not interchangeable
- @Value
  - For isolated, individual properties
  - Injects a single property value by key
  - Works for a single standalone setting that has no logical relationship to other properties

```
@Service
public class NotificationService {

    @Value("${app.notification.max-retries}")
    private int maxRetries;  // injected from application.yml
}
```

Configuration Mechanisms

- The problems with @Value at scale
  - Property keys are scattered across many classes so that renaming a key requires a codebase-wide search
  - No type validation at startup
    - A missing or malformed value causes a runtime failure, not a startup failure
  - No IDE autocomplete inside YAML files
  - No grouping
    - Related settings are not co-located
- Use @Value only for genuinely isolated, one-off properties
  - If there are more than 3 @Value annotations for related setting
  - Code smell to use @ConfigurationProperties

```
@Service
public class NotificationService {

    @Value("${app.notification.max-retries}")
    private int maxRetries;  // injected from application.yml
}
```

Configuration Mechanisms

- @ConfigurationProperties
  - Strongly typed configuration binding
- Binds an entire subtree of the YAML configuration to a single Java class or record
  - Correct approach whenever a group of related properties belong together.

Step 1: Define the YAML structure:

```yaml
# application.yml
app:
  employee-service:
    base-url: https://hr-api.internal.example.com
    timeout-seconds: 10
    retry-attempts: 3
    circuit-breaker-threshold: 50
```

Step 2: Create a record to bind to that subtree:

```java
@ConfigurationProperties(prefix = "app.employee-service")
@Validated
public record EmployeeServiceConfig(
    @NotBlank String baseUrl,
    @Min(1) @Max(60)  int timeoutSeconds,
    @Min(1) @Max(10)  int retryAttempts,
    @Min(1) @Max(100) int circuitBreakerThreshold
) {}
```

Configuration Mechanisms

- @ConfigurationProperties
  - Strongly typed configuration binding
- Binds an entire subtree of the YAML configuration to a single Java class or record
  - Correct approach whenever a group of related properties belong together.

Step 3: Register it with the container:

```
// Add to your @SpringBootApplication class or any @Configuration class
@EnableConfigurationProperties(EmployeeServiceConfig.class)
```

Step 4: Inject it like any other bean:

```
@Service
public class EmployeeApiClient {

    private final EmployeeServiceConfig config;

    public EmployeeApiClient(EmployeeServiceConfig config) {
        this.config = config;
    }

    public Employee fetchEmployee(Long id) {
        // config.baseUrl(), config.timeoutSeconds() etc. are available here
    }
}
```

Configuration Mechanisms

- @ConfigurationProperties(prefix = "app.employee-service")
  - Binds the YAML subtree starting at app.employee-service to the fields of this record
  - Spring handles the relaxed binding automatically
    - Ttmeout-seconds in YAML binds to timeoutSeconds in Java
    - TIMEOUT_SECONDS as an environment variable also binds to timeoutSeconds.
    - One field, multiple valid source formats
  - Java record immutability
    - Once the application starts the configuration values cannot be changed.
    - An injected configuration record is a guaranteed-stable set of values for the lifetime of the bean that holds it

Step 1: Define the YAML structure:

```yaml
# application.yml
app:
  employee-service:
    base-url: https://hr-api.internal.example.com
    timeout-seconds: 10
    retry-attempts: 3
    circuit-breaker-threshold: 50
```

Step 2: Create a record to bind to that subtree:

```java
@ConfigurationProperties(prefix = "app.employee-service")
@Validated
public record EmployeeServiceConfig(
    @NotBlank String baseUrl,
    @Min(1) @Max(60)  int timeoutSeconds,
    @Min(1) @Max(10)  int retryAttempts,
    @Min(1) @Max(100) int circuitBreakerThreshold
) {}
```

Configuration Mechanisms

- @Validated
  - Used on a @ConfigurationProperties class activates startup validation specifically
  - Without @Validated Spring populates the record from the YAML and environment variables and never checks the constraints
  - @Validated is the instruction to Spring Boot to run the Bean Validation engine against it before allowing the application to finish starting
  - Dependency is specified in the pom.xml

```
// @Validated on a @ConfigurationProperties record — validates at startup
@ConfigurationProperties(prefix = "app.employee-service")
@Validated
public record EmployeeServiceConfig(...) {}
```

```
APPLICATION FAILED TO START
***************************
Description:
Binding to target org.springframework.boot.context.properties.bind.BindException:
  Failed to bind properties under 'app.employee-service' to EmployeeServiceConfig

Reason: timeoutSeconds: must be between 1 and 60
```

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Spring Profiles

- Spring Profile
  - Named tag that groups beans and configuration together.
    - When a profile is active, everything tagged with that profile is loaded
    - When it is not active, everything tagged with it is ignored.
  - Provides a structured way to vary behaviour across environments without branching in code
  - Profiles activate configuration files
  - Spring Boot looks for files matching the pattern application-{profile}.yml
    - If the dev profile is active, it loads application.yml first (always loaded)
    - Then overlays application-dev.yml on top of it
    - Values in the profile-specific file override values in the base file

```
Active profile: dev
──────────────────────────────────────────────────
Loaded: application.yml          (base defaults)
Loaded: application-dev.yml      (dev overrides)

Active profile: prod
──────────────────────────────────────────────────
Loaded: application.yml          (base defaults)
Loaded: application-prod.yml     (prod overrides)
```

```java
// This bean is only created when the 'dev' profile is active
@Component
@Profile("dev")
public class DevDataSeeder implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) {
        // Load test data at startup — only in dev
    }
}
```

Spring Profiles

- Activating profiles
  - The mechanism for activating a profile depends on the current context
- Container or CI/CD pipeline
  - OS environment variable
  - The container platform sets this variable
  - The application reads it at startup
  - Production-standard approach
  - The running environment controls its own configuration without the application needing to know where it is deployed

```
SPRING_PROFILES_ACTIVE=prod
```

Spring Profiles

- During local development
  - Command-line argument
  - Or via an IntelliJ Run Configuration
  - Set Active Profiles to dev in the Spring Boot run configuration panel
- Annotation
  - Shown in the example
- Composing multiple profiles:
  - Multiple profiles can be active simultaneously
    - Each profile file contributes its overrides
    - Later profiles in the list take precedence over earlier ones when the same property is defined in multiple profile files

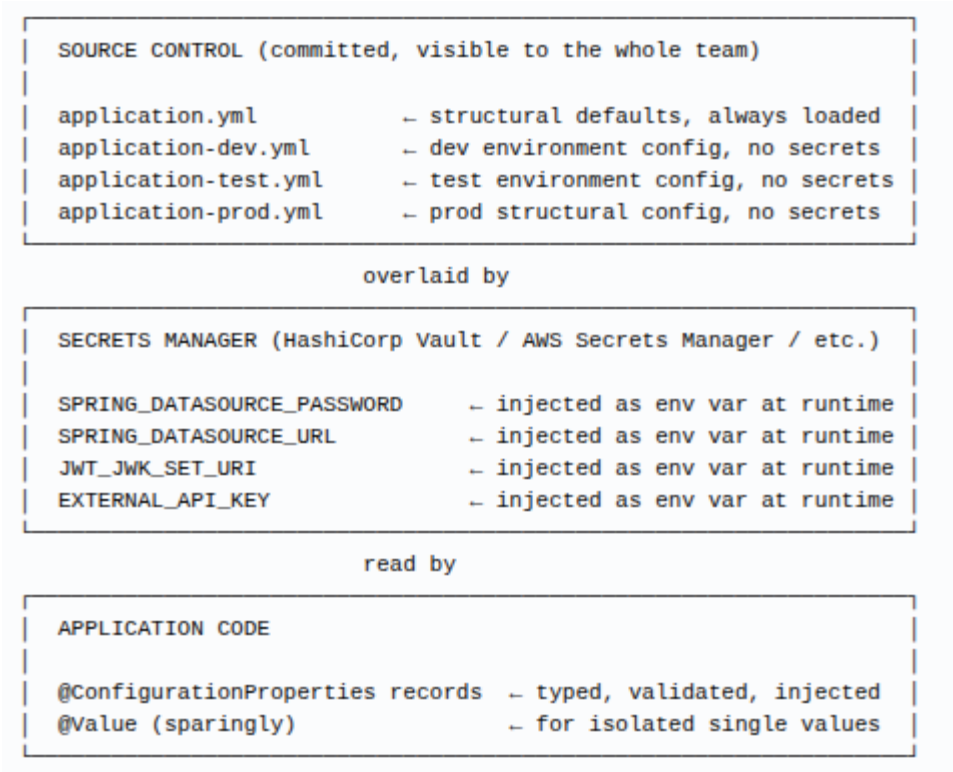
```
java -jar employee-service.jar --spring.profiles.active=dev
```

```
@SpringBootTest
@ActiveProfiles("test")
class EmployeeServiceIntegrationTest {
    // loads application.yml + application-test.yml
    // typically points to an in-memory H2 database
}
```

```
SPRING_PROFILES_ACTIVE=prod,kafka-disabled,feature-new-payroll
```

Configuration Strategy

- The invariants of this strategy
  - The same JAR file is deployed to every environment
  - No secrets in any file that touches source control
  - Misconfiguration is a startup failure, not a runtime failure
  - Every configuration group has one place to look
    - The @ConfigurationProperties record

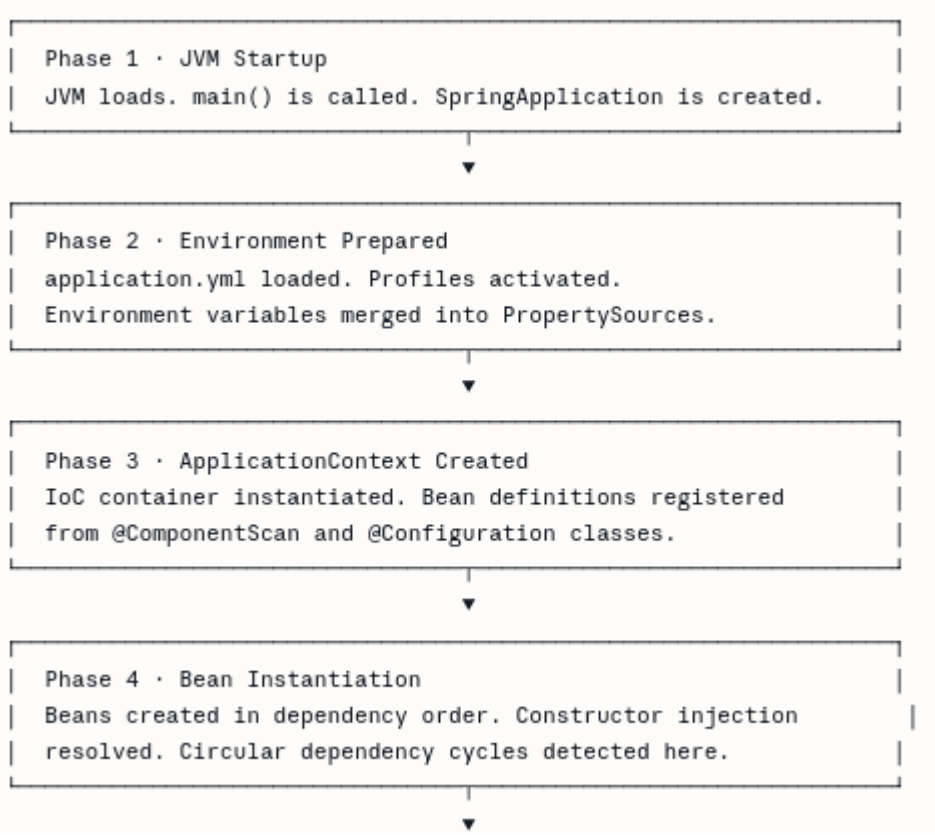


Common Configuration Mistakes

Mistake	Consequence
Hardcoding a database URL or credential in Java source	Credential is in source control forever — rotation requires a code change and redeploy
Putting a secret in `application-prod.yml` and committing it	Secret is in git history permanently, even after deletion
Using `@Value` for a group of related properties	Refactoring a property key requires searching the entire codebase; no startup validation
Omitting `@Validated` from `@ConfigurationProperties`	Invalid configuration values (wrong type, blank required field) cause a runtime failure during a live request
Setting `SPRING_PROFILES_ACTIVE=prod` in `application.yml`	The application always runs in prod mode regardless of the actual environment — test environments use production configuration
Exposing `/actuator/env` in production	The actuator endpoint reveals all resolved property values, including secrets injected as environment variables

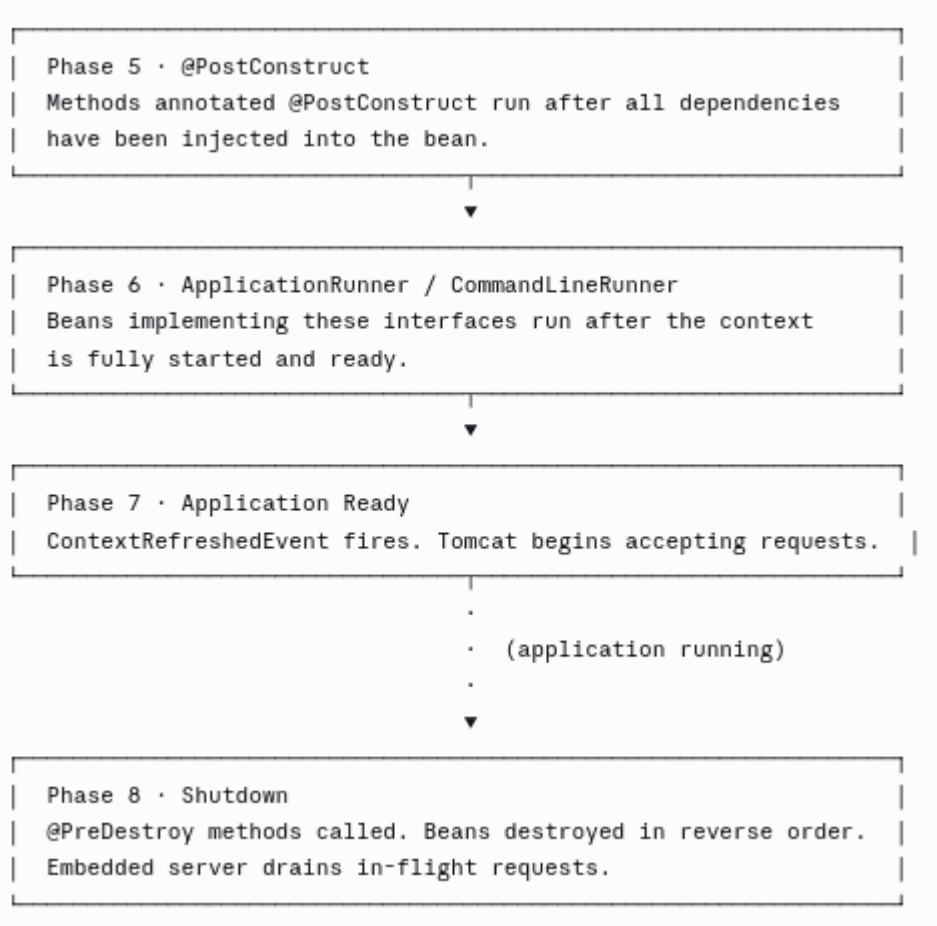
Spring Boot Startup Lifecycle

- Understanding what Spring Boot does when an application starts helps
- Explain ordering failures
  - A bean trying to use another bean or a configuration value that is not yet available at the point it initializes
  - Only appears at startup
  - Produces confusing error messages
  - Difficult to diagnose without a mental model of what is happening.



Spring Boot Startup Lifecycle

- Understanding what Spring Boot does when an application starts helps
- Identifies the correct extension points
  - Applications need to perform work at startup
    - Warming a cache
    - Verifying a database connection
    - Seeding test data
    - Registering with a service registry.
  - Spring Boot provides specific hooks for each of these needs
  - Using the wrong hook produces fragile code that fails under certain conditions



Before the Container Exists

- 1: JVM startups
  - This is the entry point provided in the source code
  - Every Spring Boot application has a main() method that hands control to the framework
- SpringApplication.run() is where Spring Boot takes over
  - Creates a SpringApplication instance
  - Determines what kind of application this is (web, reactive, or non-web)
  - Begins the startup sequence.

```
@SpringBootApplication
public class EmployeeServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(EmployeeServiceApplication.class, args);
    }
}
```

Before the Container Exists

- 2: Environment prepared
  - Before any beans are created, Spring Boot assembles the complete configuration environment.]
  - This is when
    - Application.yml is read from the classpath
    - application-{profile}.yml files are read for any active profiles
    - OS environment variables are merged in (at higher precedence)
    - Command-line arguments are merged in (at highest precedence)
  - Produces a fully resolved Environment object
    - A single view of all properties from all sources, with the precedence rules already applied.
  - By the time beans are created in phase 4, all configuration is already resolved
  - A bean's constructor can then safely read configuration values because the environment was fully assembled two phases earlier

Container Is Built But Empty

- 3: ApplicationContext created
  - The IoC container is instantiated here
  - At this point it exists but contains no live beans yet
  - It only contains bean definitions
- Builds complete set of bean definitions
  - Scans the package tree for stereotype annotations and processing @Configuration classes
  - Every class annotated with @Component, @Service, @Repository, @Controller, or @Configuration becomes a bean definition in the registry

```
Phase 3: Bean definition registry
──────────────────────────────────
EmployeeController   → definition registered (not yet instantiated)
EmployeeService      → definition registered (not yet instantiated)
EmployeeRepository   → definition registered (not yet instantiated)
EmployeeServiceConfig → definition registered (not yet instantiated)
```

Beans Are Created

- 4: Bean instantiation
  - Container works through the bean definition registry and creates instances
  - Resolves the dependency graph
    - If EmployeeController depends on EmployeeService which depends on EmployeeRepository
    - The repository is created first, then the service (with the repository injected), then the controller (with the service injected)
  - With constructor injection, this phase is where all dependency wiring happens
  - Circular dependency cycle detection
    - The container detects the cycle and refuses to start.

```
// When the container instantiates EmployeeService in phase 4:
// 1. It looks at the constructor parameters
// 2. It finds (or creates) an EmployeeRepository bean
// 3. It calls: new EmployeeService(employeeRepository)
// 4. The Objects.requireNonNull() calls fire immediately
// 5. If any dependency is missing, startup fails here with a clear message

@Service
public class EmployeeService {

    private final EmployeeRepository repository;

    public EmployeeService(EmployeeRepository repository) {
        this.repository = Objects.requireNonNull(repository);
    }
}
```

Post-Construction Initialization

- 5: @PostConstruct
  - After a bean has been fully constructed and all its dependencies have been injected
  - Spring calls any method annotated with @PostConstruct
    - This gives a bean the opportunity to perform initialization that requires its dependencies to already be present
    - Something a constructor cannot do safely if the initialization work involves calling injected collaborators
  - For internal bean setup that requires injected dependencies
    - Initializing a data structure, registering a listener on an injected event bus, computing a derived value from injected configuration

```java
@Service
public class CurrencyRateCache {

    private final CurrencyRateRepository repository;
    private Map<String, BigDecimal> rates;

    public CurrencyRateCache(CurrencyRateRepository repository) {
        this.repository = repository;
        // Cannot call repository.findAll() here safely during construction
    }

    @PostConstruct
    public void initialise() {
        // Safe here — repository is fully initialised and injected
        // But see the warning below about what NOT to do
        this.rates = new HashMap<>();
    }
}
```

Post-Construction Initialization

- 5: @PostConstruct
  - Not to be used for
    - External I/O, database queries, HTTP calls
    - Any work that depends on the full application being ready
    - The full application context is not yet ready at phase 5.
    - Other beans may still be initializing Tomcat is not yet listening

The specific failure mode to avoid:

```
@PostConstruct
public void initialise() {
    // WRONG — this calls another service that may not yet be initialised
    List<Employee> employees = employeeService.findAll();

    // WRONG — this makes an HTTP call before the application is ready
    externalApiClient.verifyConnectivity();
}
```

Use `ApplicationRunner` (phase 6) for any of this work.

Startup Work

- 6: ApplicationRunner / CommandLineRunner
- At this point:
  - Every bean has been created and fully initialized
  - All @PostConstruct methods have completed
  - The embedded Tomcat server is listening for requests
  - The application context is fully ready
  - This the correct and safe location for any real work that needs to happen at startup
    - Seeding a database, warming a cache by querying it, verifying connectivity to an external service, or running a schema validation check
  - ApplicationRunner is the preferred interface
  - Implement it on any @Component or @Service and Spring will call run() automatically in phase 6

```java
@Component
public class StartupConnectivityCheck implements ApplicationRunner {

    private final ExternalPaymentApiClient paymentApiClient;
    private final CacheWarmingService cacheWarmingService;

    public StartupConnectivityCheck(
            ExternalPaymentApiClient paymentApiClient,
            CacheWarmingService cacheWarmingService) {
        this.paymentApiClient = paymentApiClient;
        this.cacheWarmingService = cacheWarmingService;
    }

    @Override
    public void run(ApplicationArguments args) {
        // Safe — everything is ready
        paymentApiClient.verifyConnectivity();   // throws if unreachable
        cacheWarmingService.warmEmployeeCache();  // queries the database
    }
}
```

Startup Work

- 6: ApplicationRunner
  - If run() throws an exception, the application shuts down
    - This is the desired behaviour
    - An application that cannot reach a critical dependency at startup should not start at al
  - CommandLineRunner is an older equivalent
    - Receives raw String[] args instead of a typed ApplicationArguments object
    - Prefer ApplicationRunner in new code.

```
@Component
public class StartupConnectivityCheck implements ApplicationRunner {

    private final ExternalPaymentApiClient paymentApiClient;
    private final CacheWarmingService cacheWarmingService;

    public StartupConnectivityCheck(
            ExternalPaymentApiClient paymentApiClient,
            CacheWarmingService cacheWarmingService) {
        this.paymentApiClient = paymentApiClient;
        this.cacheWarmingService = cacheWarmingService;
    }

    @Override
    public void run(ApplicationArguments args) {
        // Safe — everything is ready
        paymentApiClient.verifyConnectivity();   // throws if unreachable
        cacheWarmingService.warmEmployeeCache();  // queries the database
    }
}
```

Ready and Shutdown

- 7: Application ready
  - ContextRefreshedEvent fires, signalling that the context is fully refreshed and the application is ready
  - The moment the application transitions from starting to serving

```java
// Rarely needed — prefer ApplicationRunner for startup work
@Component
public class PostStartupNotifier implements ApplicationListener<ContextRefreshedEvent> {

    @Override
    public void onApplicationEvent(ContextRefreshedEvent event) {
        // Fires when context is fully refreshed
        // Note: can fire more than once in some contexts — guard against this if needed
    }
}
```

Ready and Shutdown

- 8: Shutdown
  - When the JVM receives a shutdown signal Spring Boot begins an orderly shutdown
    - The embedded Tomcat stops accepting new requests
    - In-flight requests are allowed to complete (graceful shutdown)
    - @PreDestroy methods are called on beans in reverse creation order
    - The JVM exits
  - @PreDestroy is where you release resources that were acquired during the bean's lifetime

```
@Component
public class ManagedConnectionPool {

    private final ConnectionPool pool;

    public ManagedConnectionPool(ConnectionPool pool) {
        this.pool = pool;
    }

    @PreDestroy
    public void shutdown() {
        pool.close();           // release database connections
        pool.awaitTermination(); // wait for in-flight queries to complete
    }
}
```

Ready and Shutdown

- 8: Shutdown
  - Graceful shutdown must be explicitly enabled in Spring Boot 2.3+

```yaml
server:
  shutdown: graceful            # wait for in-flight requests before stopping

spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s  # maximum wait before forcing shutdown
```

The Service Locator Anti-Pattern

- Reaching into the ApplicationContext directly to fetch a bean as shown
  - It defeats the entire purpose of dependency injection
  - Instead of the container knowing what EmployeeService needs and providing it
  - EmployeeService is now reaching into the container to find things itself
    - The dependencies are hidden
    - Cannot know what EmployeeService requires by reading its constructor
    - The class cannot be tested without a real ApplicationContext
    - The runtime getBean() call will fail with an opaque error if the bean does not exist

```
@Service
public class EmployeeService {

    @Autowired
    private ApplicationContext context;  // injecting the container itself

    public void process() {
        // Fetching a bean manually from the container at runtime
        AuditService audit = context.getBean(AuditService.class);
        audit.record(...);
    }
}
```

The Service Locator Anti-Pattern

- The correct version

```
@Service
public class EmployeeService {

    // Dependency declared explicitly — container provides it
    private final AuditService auditService;

    public EmployeeService(AuditService auditService) {
        this.auditService = auditService;
    }

    public void process() {
        auditService.record(...);  // always available, always non-null
    }
}
```

Extension Point: A Decision Guide

What you need to do	Correct extension point	Why
Modify the property sources before the container starts	`EnvironmentPostProcessor`	Runs in phase 2 before any beans exist
Register beans programmatically based on a condition	`@Import` + conditional `@Configuration`	Runs in phase 3 during bean definition registration
Validate that required dependencies are non-null	Constructor with `Objects.requireNonNull()`	Runs in phase 4 at the earliest possible moment
Initialise internal state that needs injected collaborators	`@PostConstruct`	Runs in phase 5 after all dependencies are injected
Query the database at startup	`ApplicationRunner`	Runs in phase 6 after the full context is ready
Warm a cache at startup	`ApplicationRunner`	Runs in phase 6 after the full context is ready
Verify connectivity to an external service	`ApplicationRunner`	Runs in phase 6; failure here prevents the app from starting
Release database connections or thread pools on shutdown	`@PreDestroy`	Runs in phase 8 in reverse creation order
Enable graceful request draining during rolling deployments	`server.shutdown: graceful` in `application.yml`	Configures the embedded server's shutdown behaviour

Engineering Principles

Convention	Engineering Principle	Banking Context Benefit
Constructor injection with final fields	Explicit dependencies, immutability, fail-fast construction	Security beans are guaranteed non-null; configuration errors surface at deployment, not in production transactions.
Service layer owns @Transactional	Single responsibility; transaction boundary separated from HTTP concern	Transaction rollback on business rule violations, not on JSON marshalling errors.
DTO boundary between controller and domain	API contract isolation, information hiding	Database schema changes do not break API consumers; no accidental exposure of sensitive entity fields.
@ConfigurationProperties over @Value	Type safety, cohesion, startup validation	Misconfiguration crashes at startup. Silent misconfiguration in production is a compliance risk.
Secrets via environment variables, not YAML	Least privilege, environment parity, secret hygiene	Credentials never appear in source control, build artefacts, or logs.
Actuator health and metrics from day one	Observability as a first-class requirement	Services are compatible with orchestration, monitoring dashboards, and operational compliance controls from their first deployment.

Questions?